

SOC ANALYST PORTFOLIO CASE STUDY

Investigating Network Reconnaissance with Wireshark and Wazuh

Packet analysis, service enumeration, UFW telemetry, and multi-source SOC correlation

RYAN BRYANT | SOC / CYBERSECURITY ANALYST CANDIDATE | AUGUST 2026

OUTCOME Captured and analyzed 174 packets, identified a 13-port TCP reconnaissance pattern, confirmed SSH/22 and HTTP/80 as reachable services, extracted OpenSSH and Apache service banners, deployed a Wazuh endpoint agent, and correlated UFW Level 7 alerts with the observed network activity.

10.10.10.3 Wazuh / Probe Host → SOC-Lab 10.10.10.0/24 → 10.10.10.4 LINUX-WEBSERVER01

Network evidence: tcpdump / Wireshark | Host evidence: UFW / Wazuh

Portfolio note: Authorized local simulation only. No external or production system was targeted.
AI assistance is disclosed within this report.

1. Executive Summary

I investigated reconnaissance-like TCP traffic against LINUX-WEBSERVER01 in an isolated home SOC lab. I captured traffic on the Ubuntu endpoint with tcpdump, transferred the PCAP to Windows, analyzed the traffic in Wireshark, and then correlated the network evidence with endpoint/firewall telemetry collected by Wazuh.

The capture shows 10.10.10.3 probing 13 common service ports on 10.10.10.4. TCP/22 and TCP/80 returned SYN/ACK responses. Subsequent interaction disclosed an OpenSSH 9.6p1 Ubuntu banner and an Apache/2.4.58 (Ubuntu) Server header. Wazuh independently recorded repeated UFW blocked inbound TCP connection events from the same source to the monitored endpoint.

The evidence supports successful reconnaissance and service enumeration. It does not demonstrate successful authentication, exploitation, privilege escalation, persistence, or unauthorized system access.

Project dimension	Implementation
Role performed	Packet analyst, Tier 1 SOC analyst, and endpoint/SIEM investigator
Network evidence	174-packet PCAP analyzed in Wireshark
Observed activity	13-port TCP service discovery from 10.10.10.3 to 10.10.10.4
Reachable services	TCP/22 SSH and TCP/80 HTTP
Enumeration result	OpenSSH and Apache/Ubuntu service information disclosed
Disposition	Successful reconnaissance/enumeration; no compromise observed

2. Objectives

- Capture controlled reconnaissance-like traffic on an authorized Linux endpoint.
- Use Wireshark to identify the source, target, destination ports, TCP flags, and reachable services.
- Determine whether service interaction exposed useful banner or application information.
- Deploy and validate Wazuh endpoint telemetry before correlating host evidence.
- Distinguish successful enumeration from successful exploitation or compromise.
- Document an evidence-based severity assessment and defensive response.

3. Lab Scope and Controls

Control	Implementation
Authorized systems	Wazuh and Ubuntu VMs in the personal home lab
Network	SOC-Lab NAT network 10.10.10.0/24
Probe host	10.10.10.3 - Wazuh appliance
Monitored endpoint	10.10.10.4 - LINUX-WEBSERVER01
Capture interface	enp0s3 on LINUX-WEBSERVER01
Evidence	soc-investigation.pcap - 174 packets
Telemetry	UFW/kernel logs through Wazuh agent 001

4. Implementation Workflow

01 BASELINE

Validated connectivity and identified enp0s3 as the Linux capture interface.

02 PACKET CAPTURE

Started tcpdump on LINUX-WEBSERVER01 and recorded the test window.

03 NORMAL TRAFFIC

Generated ICMP traffic to create recognizable baseline activity.

04 CONTROLLED PROBE

Used Bash /dev/tcp attempts against 13 common TCP service ports.

05 EVIDENCE PRESERVATION

Stopped tcpdump at 174 packets and transferred the PCAP with SCP.

06 WIRESHARK TRIAGE

Identified 10.10.10.3 and 10.10.10.4 as the primary communicating hosts.

07 TCP ANALYSIS

Filtered initial SYNs and SYN/ACK responses to determine probed and reachable ports.

08 SERVICE ENUMERATION

Recovered OpenSSH and Apache/Ubuntu service information from packet contents.

09 WAZUH VALIDATION

Installed/registered Wazuh agent 001 after discovering the endpoint was not instrumented.

10 CORRELATION

Repeated a controlled probe and correlated UFW/Wazuh alerts with the network behavior.

5. Key Commands and Filters

```
ping -c 4 10.10.10.4
for port in 21 22 23 25 53 80 110 139 443 445 3306 3389 8080; do timeout 1 bash -c "echo >/dev/tcp/10.10.10.4/$port"
2>/dev/null; done
scp wazuh-user@192.168.56.101:/home/wazuh-user/soc-investigation.pcap .
tcp.flags.syn == 1 && tcp.flags.ack == 0 && ip.src == 10.10.10.3 && ip.dst == 10.10.10.4
tcp.flags.syn == 1 && tcp.flags.ack == 1 && ip.src == 10.10.10.4 && ip.dst == 10.10.10.3
tcp.port == 22 && ip.addr == 10.10.10.3 && ip.addr == 10.10.10.4
tcp.port == 80 && ip.addr == 10.10.10.3 && ip.addr == 10.10.10.4
```

SAFETY BOUNDARY All probing targeted one authorized VM on an isolated private lab network. The activity was generated only to create evidence for defensive analysis.

6. Evidence: TCP Reconnaissance

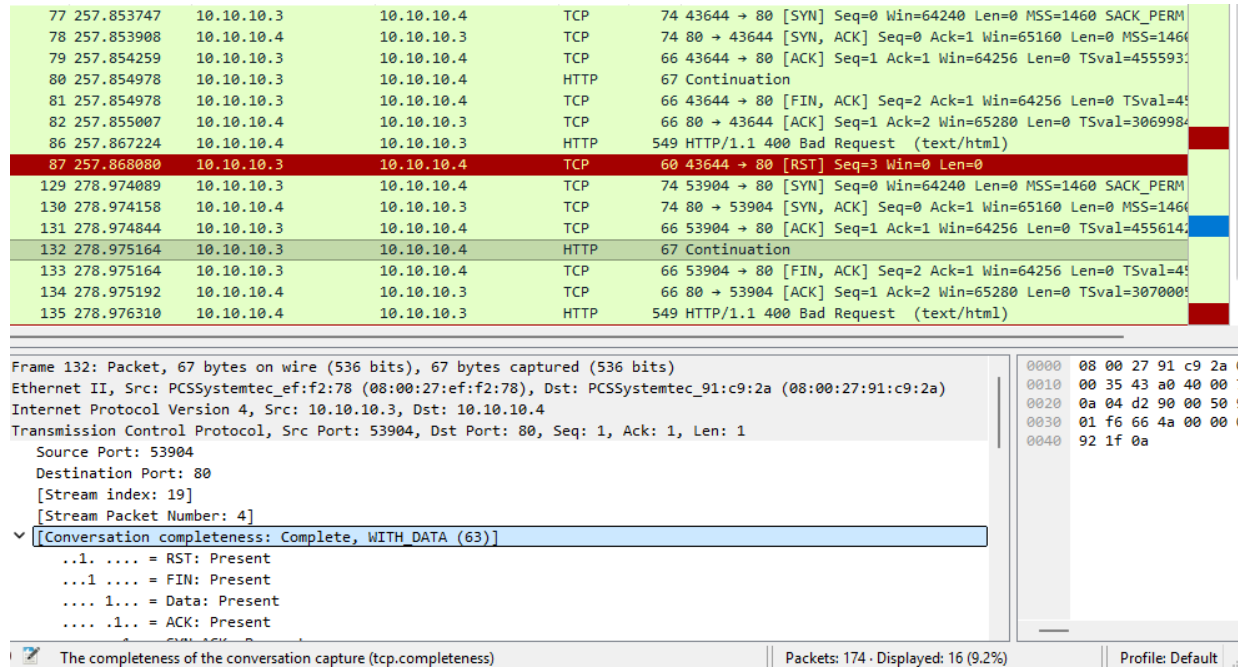


Figure 1. Wireshark TCP/HTTP investigation view from the captured traffic.

Observed destination port	Common service
21	FTP
22	SSH
23	Telnet
25	SMTP
53	DNS
80	HTTP
110	POP3
139	NetBIOS
443	HTTPS
445	SMB
3306	MySQL
3389	RDP
8080	HTTP alternate

ANALYST OBSERVATION The destination ports were common service ports and were contacted in rapid succession by the same source. This pattern is consistent with network service discovery rather than ordinary single-service application traffic.

7. Evidence: Reachable Services and SSH Enumeration

Wireshark SYN/ACK filtering showed that only TCP/22 and TCP/80 responded as reachable services in the controlled probe sequence. Verified SYN/ACK frames were 48 and 111 for SSH and 78 and 130 for HTTP.

SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.18

SSH finding	Observed value
Service	SSH on TCP/22
Handshake	SYN → SYN/ACK → ACK completed
Banner frames	65 and 119
Implementation	OpenSSH 9.6p1
Platform clue	Ubuntu packaging identifier
Authentication	No successful login demonstrated by the capture
ANALYST JUDGMENT Port 22 itself is not a vulnerability. The risk comes from exposing a remote-access service that may be misconfigured, weakly authenticated, outdated, or otherwise exploitable. The capture proves reachability and banner disclosure, not compromise.	

8. Evidence: HTTP Enumeration

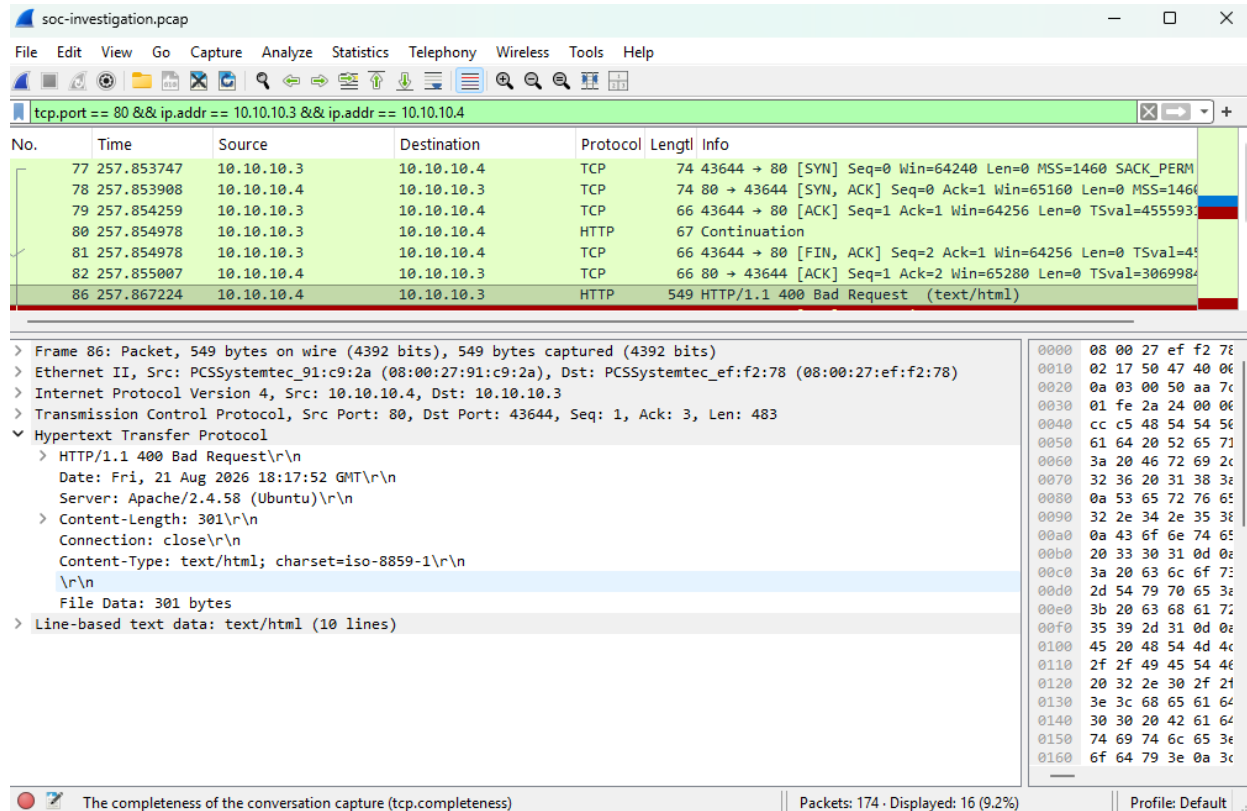


Figure 3. HTTP 400 response showing the Apache/2.4.58 (Ubuntu) Server header.

HTTP/1.1 400 Bad Request
 Server: Apache/2.4.58 (Ubuntu)
 Content-Length: 301
 Connection: close
 Content-Type: text/html; charset=iso-8859-1

HTTP finding	Observed value
Service	HTTP on TCP/80
Handshake	TCP three-way handshake completed
Response frames	86 and 135
HTTP status	400 Bad Request
Server header	Apache/2.4.58 (Ubuntu)
Interpretation	Malformed probe data was rejected at the application layer after successful TCP connectivity

KEY CORRECTION HTTP 400 does not mean TCP/80 rejected the connection. Apache received the connection and returned an application-layer response, which also disclosed useful service information.

9. Wazuh Endpoint Instrumentation

The first Wazuh review showed that no endpoint agent was registered. I did not treat unrelated manager alerts as evidence from LINUX-WEBSEVER01. Instead, I corrected the telemetry gap before continuing the investigation.

```
ocstudent@linux-webserver01:~$ sudo systemctl status wazuh-agent
wazuh-agent.service - Wazuh agent
Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
Active: active (running) since Fri 2026-08-21 20:01:33 UTC; 13s ago
Process: 3794 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
Tasks: 29 (limit: 2263)
Memory: 29.0M (peak: 36.6M)
CPU: 1.516s
CGroup: /system.slice/wazuh-agent.service
├─3817 /var/ossec/bin/wazuh-execd
├─3828 /var/ossec/bin/wazuh-agentd
├─3841 /var/ossec/bin/wazuh-syscheckd
├─3854 /var/ossec/bin/wazuh-logcollector
└─3871 /var/ossec/bin/wazuh-modulesd

ug 21 20:01:25 linux-webserver01 systemd[1]: Starting wazuh-agent.service - Wazuh agent...
ug 21 20:01:25 linux-webserver01 env[3794]: Starting Wazuh v4.14.7...
ug 21 20:01:26 linux-webserver01 env[3794]: Started wazuh-execd...
ug 21 20:01:27 linux-webserver01 env[3794]: Started wazuh-agentd...
ug 21 20:01:28 linux-webserver01 env[3794]: Started wazuh-syscheckd...
ug 21 20:01:29 linux-webserver01 env[3794]: Started wazuh-logcollector...
ug 21 20:01:31 linux-webserver01 env[3794]: Started wazuh-modulesd...
ug 21 20:01:33 linux-webserver01 env[3794]: Completed.
ug 21 20:01:33 linux-webserver01 systemd[1]: Started wazuh-agent.service - Wazuh agent.
ocstudent@linux-webserver01:~$
```

Figure 4. Wazuh agent service active and running on LINUX-WEBSEVER01.

Agents (1) ⊕ Deploy new agent ↻ Refresh 📄 Export formatted More ▾ ⚙️									
status=active									WQL
☐	ID ↑	Name	IP address	Group(s)	Operating system	Cluster node	Version	Status	Actions
☐	001	LINUX-WEBSEVER01	10.10.10.4	default	Ubuntu 24.04.4 LTS	node01	v4.14.7	● ⓘ 👁️ ⋮	

Rows per page: 10 ▾ ◀ 1 ▶

Figure 5. Wazuh dashboard showing LINUX-WEBSEVER01 (agent 001) active at 10.10.10.4.

Agent field	Observed value
Agent ID	001
Name	LINUX-WEBSEVER01
IP	10.10.10.4
Operating system	Ubuntu 24.04.4 LTS
Architecture	x86_64
Wazuh version	4.14.7
Status	Active

TELEMETRY LESSON Before trusting SIEM results, verify that the endpoint is actually instrumented and reporting. An absence of alerts is not meaningful when the endpoint agent is missing.

10. Evidence: Wazuh / UFW Correlation

296 hits

Aug 20, 2026 @ 16:12:23.526 - Aug 21, 2026 @ 16:12:23.526

Export Formatted Reset view 638 available fields Columns Density 1 fields sorted Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 21, 2026 @ 16:10:48.5...	LINUX-WEBSERVER01	UFW blocked inbound n...	7	100100
Aug 21, 2026 @ 16:10:44.5...	LINUX-WEBSERVER01	UFW blocked inbound n...	7	100100
Aug 21, 2026 @ 16:10:44.5...	LINUX-WEBSERVER01	UFW blocked inbound n...	7	100100
Aug 21, 2026 @ 16:10:34.5...	LINUX-WEBSERVER01	UFW blocked inbound n...	7	100100
Aug 21, 2026 @ 16:10:34.5...	LINUX-WEBSERVER01	UFW blocked inbound n...	7	100100
Aug 21, 2026 @ 16:10:32.5...	LINUX-WEBSERVER01	UFW blocked inbound n...	7	100100
Aug 21, 2026 @ 16:10:30.5...	LINUX-WEBSERVER01	UFW blocked inbound n...	7	100100
Aug 21, 2026 @ 16:10:30.4...	LINUX-WEBSERVER01	UFW blocked inbound n...	7	100100
Aug 21, 2026 @ 16:10:28.5...	LINUX-WEBSERVER01	UFW blocked inbound n...	7	100100
Aug 21, 2026 @ 16:10:28.5...	LINUX-WEBSERVER01	UFW blocked inbound n...	7	100100
Aug 21, 2026 @ 16:10:26.5...	LINUX-WEBSERVER01	UFW blocked inbound n...	7	100100
Aug 21, 2026 @ 16:02:02.6...	LINUX-WEBSERVER01	SCA summary: CIS Ubu...	5	19003
Aug 21, 2026 @ 16:01:55.5...	LINUX-WEBSERVER01	CIS Ubuntu Linux 24.04 ...	3	19008
Aug 21, 2026 @ 16:01:55.5...	LINUX-WEBSERVER01	CIS Ubuntu Linux 24.04 ...	3	19008
Aug 21, 2026 @ 16:01:55.5...	LINUX-WEBSERVER01	CIS Ubuntu Linux 24.04 ...	3	19008

Rows per page: 15 < 1 2 3 4 5 ... 20 >

Figure 6. Wazuh Events view showing repeated UFW blocked inbound network connection alerts.

Alert field	Observed value
Agent	LINUX-WEBSERVER01 (001)
Rule	100100 - UFW blocked inbound network connection
Rule level	7
Decoder / program	kernel / kernel
Source / destination	10.10.10.3 / 10.10.10.4
Protocol / flag	TCP / SYN
Example destination port	21
Behavior	Repeated blocked inbound attempts during the controlled test window

10.1 Expanded Event Detail

Table	JSON
<code>_index</code>	wazuh-alerts-4.x-2026.08.21
<code>agent.id</code>	001
<code>agent.ip</code>	10.10.10.4
<code>agent.name</code>	LINUX-WEBSEVER01
<code>decoder.name</code>	kernel
<code>full_log</code>	Aug 21 20:11:24 server01 kernel: [UFW BLOCK] IN=enp0s3 OUT= MAC=08:00:27:91:c9:2a:08:00:27:ef:f2:78:08:00 SRC=10.10.10.3 DST=10.10.10.4 LEN=60 TOS=0x00 PREC=0x00 TTL=127 ID=24583 DF PROTO=TCP SPT=60990 DPT=110 WINDOW=64240 RES=0x00 SYN URGP=0
<code>id</code>	1787343048.1252618
<code>input.type</code>	log
<code>location</code>	journald
<code>manager.name</code>	wazuh-server
<code>predecoder.hostname</code>	linux-webserver01
<code>predecoder.program_name</code>	kernel
<code>predecoder.timestamp</code>	Aug 21 20:11:24
<code>rule.description</code>	UFW blocked inbound network connection.
<code># rule.firedtimes</code>	22

Figure 7. Expanded Wazuh event showing UFW BLOCK metadata from 10.10.10.3 to 10.10.10.4.

CORRELATION POINT The expanded UFW record identifies the same source and destination relationship observed in Wireshark and records TCP SYN activity on the monitored endpoint.

11. Evidence Correlation and Final Disposition

Question	Assessment
What happened?	10.10.10.3 probed multiple common TCP service ports on 10.10.10.4.
What did Wireshark prove?	13 destination ports were probed; SSH/22 and HTTP/80 were reachable and disclosed service information.
What did Wazuh prove?	The endpoint generated repeated UFW BLOCK records for inbound TCP SYN activity from 10.10.10.3.
Was enumeration successful?	Yes. The source identified reachable services and obtained OpenSSH and Apache/Ubuntu information.
Was compromise successful?	No compromise was observed. No successful SSH authentication, exploitation, persistence, or unauthorized control was demonstrated.
Suggested severity	Low-to-Medium in this lab context; raise severity if the source is unauthorized/external or follow-on exploitation occurs.
Classification	True positive reconnaissance/enumeration activity generated in an authorized lab.
FINAL DISPOSITION SUCCESSFUL RECONNAISSANCE / ENUMERATION — NO SYSTEM COMPROMISE OBSERVED.	

Wireshark and Wazuh provided complementary visibility. Wireshark exposed handshakes, application responses, resets, and service banners. Wazuh exposed what the endpoint logged and what its detection rules classified. Correlating both sources produced a stronger incident narrative than either source alone.

12. Recommended Response in a Real Environment

- Validate whether the scanning source is authorized and identify the asset owner and business purpose of the target.
- Review whether SSH must be reachable from the observed network segment; restrict access with firewall rules, segmentation, or allowlists where appropriate.
- Disable direct root SSH login and prefer strong key-based authentication where operationally appropriate.
- Review SSH authentication logs for successful or failed login activity following the reconnaissance.
- Patch and harden exposed OpenSSH and Apache services based on organizational vulnerability-management policy.
- Reduce unnecessary banner/version disclosure where practical, while recognizing that banner suppression is not a substitute for patching and access control.
- Correlate firewall, authentication, web-server, endpoint, DNS, and network telemetry for follow-on exploitation indicators.
- Escalate if scanning expands across hosts, persists over time, is followed by authentication attempts, or produces exploitation indicators.

13. MITRE ATT&CK and Analyst Judgment

The observed behavior is consistent with Network Service Discovery (MITRE ATT&CK T1046) because one host systematically tested multiple service ports to determine which services were reachable. The report does not claim exploitation because the collected evidence supports discovery and enumeration only.

14. Assistance and Source Transparency

This project was completed by Ryan Bryant in his personal home lab with assistance from OpenAI ChatGPT ("Sarah"). ChatGPT helped structure the safe workflow, explain Wireshark and Wazuh observations, troubleshoot Wazuh agent deployment, refine the incident interpretation, validate the supplied PCAP, and edit this portfolio narrative. Ryan personally operated the VMs, entered commands, generated the traffic, captured the evidence, analyzed the packets and Wazuh events, and made the incident assessment. ChatGPT did not independently access or operate the lab computer.

15. Skills Demonstrated

- tcpdump packet capture and evidence preservation
- Wireshark display filtering and TCP handshake analysis
- Recognition of TCP service discovery and reconnaissance patterns
- SSH and HTTP service/banner enumeration analysis
- UFW firewall log interpretation
- Wazuh Linux agent deployment and validation
- SIEM Threat Hunting and event correlation
- Multi-source network and endpoint evidence correlation
- Evidence-based distinction between enumeration and compromise
- Incident severity assessment and defensive remediation planning

16. Employer-Facing Project Summary

RESUME BULLET Investigated controlled network reconnaissance in an isolated SOC lab using Wireshark and Wazuh: analyzed a 174-packet PCAP, identified 13 probed services, confirmed SSH/22 and HTTP/80 exposure, extracted OpenSSH/Apache banners, and correlated UFW Level 7 endpoint alerts with packet-level evidence.

Interview talking points

- How I used SYN and SYN/ACK filters to distinguish probed ports from reachable services.
- Why an HTTP 400 response still proves successful TCP connectivity and can disclose service information.
- How I separated successful reconnaissance from successful compromise.
- Why I corrected the missing Wazuh agent before trusting SIEM evidence.
- How Wireshark and Wazuh provided different but complementary perspectives on the same activity.

17. References and Validation Sources

The following sources and artifacts were used to validate the workflow and technical interpretation in this case study:

- Wireshark User's Guide and display-filter documentation - packet filtering, TCP fields, and protocol analysis.
- Wazuh documentation - Linux agent deployment, agent registration, Threat Hunting, and alert investigation.
- Ubuntu UFW documentation / manual - host-firewall behavior and logged block events.
- MITRE ATT&CK T1046 - Network Service Discovery - behavioral mapping for multi-port service discovery.
- soc-investigation.pcap - 174-frame packet capture used to verify frame numbers, TCP flags, service banners, and HTTP responses.
- Wazuh screenshots - agent status, Threat Hunting alert list, and expanded UFW event metadata.
- OpenAI ChatGPT - guided lab assistance, troubleshooting support, PCAP cross-checking, and report editing.

EVIDENCE INTEGRITY Frame numbers cited in this report are specific to soc-investigation.pcap. The supplied capture was independently checked for the 13 SYN probe destinations, SYN/ACK responses from TCP/22 and TCP/80, SSH banners in frames 65 and 119, and HTTP 400/Apache responses in frames 86 and 135.